ResumeMatch Lab — A Methodology Case Study

Designing a rigorous A/B test for resume–job match quality

ResumeMatch Lab

2026

1. Executive Summary

ResumeMatch Lab answers a deceptively simple question a job seeker actually has: *"I have two versions of my resume — which one matches more jobs, by how much, and where?"* We treat it as a designed experiment rather than a vibe check. Both resume variants are embedded with a sentence-transformer (`BAAI/bge-small-en-v1.5`, 384-dim) and scored by cosine similarity against a fixed snapshot of **2,000 real Indian tech job postings**. Because every job is scored by **both** resumes, the data is naturally **paired**, and the per-job score difference `d_i = score_B(i) - score_A(i)` becomes the unit of analysis.

On that paired-delta vector we run a deliberately over-complete battery of methods, each chosen to answer a different question:

Question	Method
Is the average difference real?	Paired t-test / Wilcoxon (normality-gated)
How big, with what uncertainty?	Cohen's d; bootstrap **BCa** 95% CI
Could we even have detected it?	Power analysis; achieved power; MDE table
Can we tighten the estimate?	**CUPED** variance reduction
Is it valid if we peek early?	**mSPRT** always-valid p-value
How probable is "B is better"?	Bayesian **Beta-Binomial** posterior
Where does the effect live?	Per-cluster tests + **Bonferroni / BH-FDR**

The key methodological choices — a paired design, a normality gate for test selection, BCa intervals, job-side CUPED covariates, a mixture sequential test, and multiplicity control across 8 job clusters — are explained and justified in the sections that follow.

Headline result on the reference run. Comparing a DevOps-flavored resume (A) against a Data-Science-flavored resume (B) over the corpus, A wins *overall* by **2.08 points** (bootstrap BCa 95% CI [1.84, 2.30], Wilcoxon p ~= 7e-75, Cohen's d = -0.40). But the global verdict is the least interesting output: the **per-cluster** breakdown shows B decisively wins **Machine Learning / AI (+3.87 pts)** and **QA / Testing (+5.24 pts)** while A dominates Data Engineering, DevOps, Backend, and the rest. The product's real recommendation is not "send A" — it is *"send B for ML and QA roles, send A for infrastructure roles."* That conditional, defensible answer is what separates this from a single-number resume scorer.

A reader with only 90 seconds should read this section and Section 12 (the decision framework). A reader checking whether the author understands the methods should read Sections 6, 8, and 9.

Why so many methods. A single test would be enough to *declare* a winner; the over-complete battery exists to make the declaration *trustworthy* and *defensible*. Each method covers a different failure mode of the others — the bootstrap drops the normality assumption, the Bayesian posterior reframes significance as probability, the per-cluster correction catches a reversed sign the global test hides, and mSPRT removes the

peeking caveat entirely. Convergent evidence across independent methods is far harder to dismiss than one p-value, and being able to explain *why each one is present* is itself the purpose of a methodology document.

2. Problem Framing

A job seeker rarely has one resume; they have variants — a "data" version and a "platform" version, a keyword-dense version and a narrative version. The decision they want to make is concrete: **which variant to send**, and whether the choice even matters. The naive approach (generic ATS keyword scoring) ignores the job market the resume is actually competing in, and reports a single number with no uncertainty.

We reframe the question as an experiment with a clear estimand: the mean difference in match score between variant B and variant A across a representative population of jobs. This framing buys us everything experimental design offers — effect sizes, confidence intervals, power, and the ability to say *"the lift is concentrated in ML/AI roles; A still wins for Backend."*

Why uncertainty is the product, not a footnote. A scorer that reports "B: 71, A: 68" invites a false decision: the gap may be pure noise. The cost of acting on noise is real — the candidate sends the wrong variant to roles they care about. By attaching a confidence interval and a per-cluster map to the verdict, we let the user distinguish three genuinely different situations: (i) B is reliably better everywhere, (ii) the two are indistinguishable and the choice does not matter, and (iii) the winner *flips* by role, so the right action is to keep both and target them. Only framing the question as an experiment surfaces case (iii), which is the most common and the most actionable.

The decision-theoretic view. Let the loss of sending the worse variant to a targeted role be some `L > 0`. A point estimate minimizes expected loss only if its sign is trustworthy; an interval that straddles zero tells the user the sign is not yet identified. The entire analytical apparatus below exists to make the sign — and its domain of validity — trustworthy, per role.

Who the user is. Three personas anchor the design (detailed in the product's persona docs): the *Fresher*, who has one resume and wants to know if a rewrite even helps; the *Switcher*, moving between role families (say, backend to data) and genuinely unsure which framing lands; and the *Senior* candidate, optimizing a strong resume at the margin. The Switcher is the sharpest case for this product — their two variants legitimately target different cluster mixes, so a per-cluster verdict is not a nicety but the core answer they came for. The same machinery serves all three, but it earns its complexity on the Switcher.

The cost of guessing. Without an experiment, the fallback is intuition: send whichever resume "feels" more polished. That is fine when the variants are interchangeable and costly when they are not — sending the platform-engineering resume to the data roles the candidate most wants is a silent, unrecoverable miss, since they never learn the rejection was avoidable. Quantifying the gap and localizing it by role converts an invisible cost into a visible, addressable decision, which is the entire justification for the machinery that follows.

3. Hypothesis and Metric Design

Hypotheses. For mean paired difference `mu_d`:

$$H_0 : \mu_d = 0 \qquad H_1 : \mu_d \neq 0$$

We test the **two-sided** alternative deliberately. A one-sided test would presuppose which variant is better, which is exactly the question the user is asking; committing to a direction in advance would also double the effective false-positive rate if the analyst peeked at the data first. The two-sided framing keeps the test honest about a possibly *negative* effect.

Primary metric. The per-job paired delta and its mean:

$$d_i = \text{score}_B(i) - \text{score}_A(i), \qquad \hat{d} = \frac{1}{N}\sum_{i=1}^{N} d_i$$

A **paired** design is the right model because the same 2,000 jobs are scored by both resumes. To see why pairing matters, decompose each score into a *job effect* `g_i` (how matchable job i is, independent of which resume scores it) plus resume-specific noise `e_{X,i}`. An unpaired two-sample comparison would carry the variance of the job effects — which is large, since jobs differ enormously in how matchable they are — straight into the standard error. Pairing differences the job effect out entirely:

$$d_i = (g_i + e_{B,i}) - (g_i + e_{A,i}) = e_{B,i} - e_{A,i},$$

so only the resume-specific noise survives. This is why the paired design is strictly more powerful here and why we never consider the unpaired alternative.

Guardrail metrics (surfaced to the user, not used to declare victory):

- *Length parity.* If `|len(A) - len(B)| / max(len(A), len(B)) > 0.5`, warn — wildly different lengths can bias embeddings, since a much longer document dilutes its own topical signal under mean-pooling.
- *Skill-set parity.* If the Jaccard similarity of extracted skill sets `< 0.3`, warn — the two documents may be different roles, not A/B variants of one, in which case the comparison is apples-to-oranges and the user should know.
- *Parse-quality floor.* If either resume parses to `< 200` characters, **refuse** to score rather than report nonsense. A garbled parse silently produces a confident, meaningless verdict; refusing is the safer default.

Guardrails are reported but never gate the verdict automatically — the user decides whether a flagged comparison is still meaningful for their case.

The estimand, stated precisely. What we estimate is the average paired effect over the job population the snapshot represents:

$$\text{ATE} = \mathbb{E}_i[\text{score}_B(i) - \text{score}_A(i)],$$

estimated by the sample mean of the per-job deltas over the 2,000 jobs. Two assumptions make this clean: each job is scored independently of the others (no interference between units), and the snapshot is a fixed, representative draw from the market we care about. The second is the load-bearing assumption and is revisited as a limitation in Section 13 — a stale or skewed snapshot shifts the population the estimand refers to, even when the arithmetic is exact.

4. Embedding-Based Scoring

We embed text with `BAAI/bge-small-en-v1.5`, a 384-dimensional sentence-transformer, and measure match by cosine similarity. Four deliberate choices matter:

1. **Model reuse / consistency.** The same model and preprocessing embed the job corpus (inherited from the sibling project *JobAtlas*) and the resumes. Identical encoder, no instruction prefix, `normalize_embeddings=True` on both sides. If the resume were embedded differently from the jobs, cosine similarity would be measuring an artifact of the preprocessing rather than semantic overlap.
2. **Why a small model.** A 384-dim model trades a little semantic nuance for speed and a tiny memory footprint, so the whole product runs free on a 1 GB host and scores a resume in roughly a second on CPU. For a *relative* comparison of two resumes against the same fixed corpus, absolute embedding quality matters far less than consistency — both variants face the identical encoder and the identical jobs, so any model bias is shared and largely differenced out, exactly as the job effect was in Section 3.

3. **Normalized vectors.** Because both sides are L2-normalized, cosine similarity reduces to a dot product:

$$\cos(u, v) = \frac{u^\top v}{\|u\|\,\|v\|} = u^\top v \quad \text{when } \|u\| = \|v\| = 1,$$

   so scoring 2 resumes against 2,000 jobs is a single `(2000x384).(384,)` matrix-vector product — milliseconds on CPU, fully deterministic.

4. **Cosine as a proxy.** Cosine similarity is a proxy for *textual* relevance, not for hiring outcomes. We state this limitation plainly (Section 13) rather than overclaim it as a callback predictor. What it does measure — semantic overlap between a resume and a job's language — is precisely the lever a candidate can pull by rewriting, which is why it is the right target for an A/B test of resume *edits*.

Does cosine actually track relevance? The proxy is only useful if higher cosine really does mean "more on-topic for this job." Two checks support it. First, by construction: a resume dense in a job's vocabulary lands nearer that job in the embedding space, which is exactly what semantic match should mean. Second, by behavior on the corpus: a data-flavored resume scores systematically higher against the Data & Analytics and ML/AI clusters than against Frontend, and the reverse holds for a frontend resume — the scores sort roles the way a human reviewer would. Both resumes are encoded with mean-pooling over tokens, so a single very long document can dilute its own topical signal; this is the mechanism behind the length-parity guardrail in Section 3. None of this makes cosine a hiring-outcome predictor — only a defensible, manipulable proxy for textual fit.

Why a bi-encoder, not a cross-encoder. A cross-encoder (or an LLM judge) that reads each resume-job pair jointly would likely score relevance more accurately, but it would require one forward pass per pair — 4,000 model calls per comparison — making the product slow, costly, and non-deterministic. The bi-encoder embeds each document once and reduces scoring to a dot product, so a full 2-resume-by-2,000-job comparison is a single matrix multiply that runs in milliseconds on a free CPU host and returns the identical result every time. For a *relative* A/B contrast on a fixed corpus, that determinism and speed matter more than the last few points of absolute accuracy a cross-encoder would add.

5. Job Clusters and Stratification

A single global verdict hides the most useful insight: a resume can win overall yet lose in the roles the candidate actually targets. This is a textbook **Simpson's-paradox** risk — an aggregate sign that reverses within subgroups. We therefore cluster the 2,000 job embeddings with **K-means (k=8)** and label clusters from their dominant titles: *Data Engineering, Data & Analytics, Machine Learning / AI, DevOps / SRE / Cloud, Backend Engineering, Frontend / Mobile, Product Management, QA / Testing.* Clusters are computed once with a fixed seed and committed alongside the snapshot for reproducibility — the same labels ship with the data so the per-cluster analysis is identical for every user and every rerun.

Choosing k. We fixed `k = 8` for interpretability and stability rather than chasing a marginally better objective value: eight groups map cleanly onto recognizable role families a candidate reasons about, and the clusters stay stable across reruns under a pinned seed. Larger k fragments coherent role families into near-duplicate clusters that are hard to label and that thin out the per-cluster sample sizes (and therefore per-cluster power); smaller k merges genuinely different roles — collapsing, say, DevOps into Backend — and re-hides exactly the role-level reversals we built stratification to expose.

Cluster sizes and per-cluster power. The eight clusters are not equal in size — on the reference corpus the Data Engineering cluster alone holds about **370** of the 2,000 jobs, while others are leaner. This matters for the per-cluster tests in Section 11: a cluster's detectable effect scales as roughly `1/sqrt n_cluster`, so a 370-job cluster resolves much smaller role-level effects than a 100-job one. We therefore read a non-significant *small* cluster as "underpowered here," not "no effect," and lean on the FDR correction (rather than Bonferroni) precisely so that genuine effects in the smaller clusters are not crushed by an overly strict family-wise threshold.

Alternatives considered. LDA topic modeling gives soft, overlapping assignments that are harder to label crisply and complicate a per-cluster paired test (a job would contribute partially to several strata). Rule-based title matching is brittle to the sheer variety of job titles ("SDE-2", "Member of Technical Staff", "Backend Wizard") and bakes in the analyst's taxonomy rather than learning it from the data. K-means on embeddings gave stable, interpretable groups directly in the same vector space we score in, and feeds cleanly into the stratified analysis of Section 11.

How the clustering is computed. K-means runs Lloyd's algorithm with k-means++ initialization on the L2-normalized embeddings. On unit vectors, squared-Euclidean distance is monotone in cosine distance — `||u - v||^2 = 2(1 - u.v)` — so partitioning in Euclidean space groups jobs by semantic similarity exactly as the cosine scoring intends, with no mismatch between how we cluster and how we score. A fixed random seed makes the assignment fully deterministic, and the resulting labels are committed alongside the snapshot, so every user and every rerun sees identical clusters and the per-cluster verdicts are reproducible.

6. Frequentist Inference

Test selection is data-driven. We run a Shapiro-Wilk normality check on the deltas (subsampled at 5,000 where N is larger, since the W statistic's sensitivity grows with N and the test becomes hypersensitive to trivial departures at large samples). If the deltas are approximately normal we use the **paired t-test**; otherwise the **Wilcoxon signed-rank** test, which is distribution-free and robust to the heavy tails that embedding deltas sometimes exhibit. Gating the choice on a diagnostic — rather than always assuming normality — is what keeps the reported p-value trustworthy across very different resume pairs.

Paired t-test statistic and parametric CI:

$$t = \frac{\hat{d}}{s_d/\sqrt{N}}, \quad \mathrm{df} = N - 1, \qquad \hat{d} \pm t_{0.975,\,N-1}\,\frac{s_d}{\sqrt{N}}$$

The Wilcoxon test instead ranks the absolute deltas, sums the ranks carrying a positive sign, and compares that signed-rank sum to its null distribution. It tests whether the *median* shift is zero and makes no normality assumption, at the cost of some efficiency when the data really are normal — an acceptable price for robustness on a metric we cannot guarantee is well-behaved.

Bootstrap CIs. Independent of the parametric assumption, we resample the paired-delta vector **10,000** times with replacement and report both the percentile interval and the **bias-corrected and accelerated (BCa)** interval. The percentile interval simply takes the 2.5th and 97.5th quantiles of the resampled means. BCa improves on it by correcting for two problems the percentile method ignores — median bias and skew — via a bias-correction term

$$\hat{z}_0 = \Phi^{-1}\Big(\tfrac{1}{B}\textstyle\sum_b \mathbf{1}[\hat{d}^*_b < \hat{d}]\Big)$$

and a jackknife acceleration `a` estimated from the leave-one-out influence values. The adjusted endpoints use percentiles shifted by `z_0` and `a` rather than the naive 2.5/97.5. The corrected lower percentile, for instance, is

$$\alpha_1 = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_{\alpha/2}}{1 - \hat{a}\,(\hat{z}_0 + z_{\alpha/2})}\right),$$

and the interval is read from the bootstrap distribution at `alpha1` and its upper counterpart. BCa is our **headline** interval because the percentile interval can be anti-conservative (too narrow) under skew, and embedding deltas are frequently skewed. On the reference run the BCa interval was [1.84, 2.30] points, comfortably excluding zero.

Statistical vs practical significance. At N = 2,000 the test is so well-powered that even a trivially small delta would register as "significant" — so significance alone is not the decision. We therefore foreground the *effect size*: the 2.08-point gap is a difference in cosine-similarity units (scaled x100 for display), and Cohen's d = -0.40 places it in the "medium" band, large enough to act on. The product never reports a p-value without its companion effect size and interval, precisely so a user is not misled into treating a statistically certain but practically negligible gap as a reason to switch resumes.

Wilcoxon mechanics. When normality is rejected, the signed-rank test ranks the absolute deltas `|d_i|`, attaches each delta's sign, and sums the ranks of the positive deltas:

$$W^+ = \sum_{i:\, d_i > 0} R_i, \qquad \mathbb{E}[W^+] = \frac{N(N+1)}{4} \text{ under } H_0,$$

comparing `W^+` to that null mean via a normal approximation at this N. Because it uses ranks rather than raw magnitudes, a handful of extreme deltas cannot drag the result around — the robustness that makes it the safe default when the normality gate fires. The cost is efficiency: when the deltas really are normal, Wilcoxon needs roughly 5% more data for the same power, a price we happily pay for not assuming a distribution we cannot guarantee.

	Paired t-test	Wilcoxon signed-rank
Tests	mean shift	median shift
Assumes	normal deltas	symmetric deltas
Robust to outliers	no	yes
Selected when	Shapiro-Wilk p > 0.05	otherwise

7. Power Analysis

With N = 2,000 paired observations, the design is highly powered. Power is the probability of rejecting `H_0` when a true effect of a given size exists; reporting it converts a non-significant result from "no effect" into the sharper "no effect *we could have seen*." We report three things:

- **Required N** for 80% power at the observed effect size. For the reference run's `d = 0.40`, the two-sided requirement `N ~= ((z_{0.975} + z_{0.80})/d)^2 = (2.80/0.40)^2 ~= 49` (~= **52** with the small-sample t-correction). In other words, 2,000 jobs is roughly **40x more** than this effect needed — the design is nowhere near power-limited.
- **Achieved power** at the observed effect. For large noncentrality, SciPy's noncentral-t saturates to NaN; we fall back to the large-sample normal approximation `power ~= Phi(|d|sqrt N - z_{1-alpha/2})`, which is effectively exact at this N and returns `power ~= 1.00` on the reference run.
- A **minimum detectable effect (MDE)** table in Cohen's d across an alpha x power grid. The paired-design MDE is `d_MDE = (z_{1-alpha/2} + z_{power})/sqrt N`; at N = 2,000:

Power \ alpha	0.01	0.05	0.10
0.80	0.076	0.063	0.056
0.90	0.086	0.072	0.065
0.95	0.094	0.081	0.074

The table makes the design's resolution explicit: at the conventional alpha = 0.05 / power = 0.80 cell, this corpus can detect an effect as small as **d ~= 0.063**. Reporting it matters most when a comparison comes back non-significant — it lets the product say "we could detect d >= 0.06 at 80% power; the observed d was 0.03," distinguishing a genuine null from an underpowered one, which a bare p-value never can.

Why the paired design is so powerful here. The standard error of a paired mean scales with `s_d = sqrt (s_A^2 + s_B^2 - 2.r.s_A.s_B)`, where `r` is the correlation between A's and B's scores across jobs. Because two variants of *the same person's* resume are highly correlated (both contain the same core skills), `r` is large and `s_d` collapses far below what an unpaired contrast would see. This is the same job-effect cancellation from Section 3, viewed through the variance: the more alike the two resumes, the tighter the test — which is exactly the regime an A/B test of resume *edits* lives in, and why even modest rewrites are detectable at this N.

Required-N derivation. Inverting the power expression gives the sample size needed to reach power `1 - beta` against effect `d` at level `alpha`:

$$N \approx \left(\frac{z_{1-\alpha/2} + z_{1-\beta}}{d} \right)^2 .$$

For the reference run's `d = 0.40` at alpha = 0.05, power 0.80, this normal approximation returns `N ~= 49`; the engine's exact noncentral-t computation reports **52** — the small handful of extra observations the t-correction adds at modest N. Either way the message is identical: a medium effect needs only dozens of jobs, so 2,000 leaves the design power-saturated, and any non-significant verdict is therefore a statement about effect *size*, never about insufficient data.

8. CUPED Variance Reduction

CUPED (Controlled-experiment Using Pre-Experiment Data; Deng et al., 2013) reduces the variance of the effect estimate by regressing out variation explained by covariates that are independent of the treatment contrast. For a single covariate `X` the adjusted metric is

$$\hat{d}_i^{\text{adj}} = d_i - \theta\,(X_i - \bar{X}), \qquad \theta^\star = \frac{\text{Cov}(d, X)}{\text{Var}(X)},$$

and the optimal `theta` is exactly the OLS slope of `d` on `X`. The adjustment is **unbiased** for `mu_d` because we subtract a mean-zero quantity (`X` is centered), and the variance of the adjusted mean is reduced by precisely the squared correlation:

$$\frac{\text{Var}(\hat{d}^{\text{adj}})}{\text{Var}(\hat{d})} = 1 - \rho_{d,X}^2, \qquad \text{so} \qquad \text{variance reduction} = 1 - \frac{\text{Var}(\hat{d}^{adj})}{\text{Var}(d)} = R^2.$$

With several covariates this generalizes to the vector form `d_i^adj = d_i - (X_i - Xbar)' b_hat`, and the variance reduction equals the multiple `R^2` of the covariate regression.

A correction to the naive covariate choice. The original project spec named *resume-level* covariates (resume length, skill density). These are **constant across the 2,000 jobs** for a fixed resume pair, so they have zero variance over the unit of analysis and cannot reduce per-job delta variance — including them does literally nothing. The unit of observation is a *job*, so the covariates must vary across jobs **and** be independent of the A-vs-B contrast (otherwise the adjustment would bias the estimate). We therefore use **job-side covariates**: cluster membership (one-hot) and job-description length. Cluster membership is the workhorse — if B systematically beats A in ML/AI but loses in Backend, cluster explains a large share of the delta variance, and residualizing on it tightens the interval. These covariates are valid because they are properties of the *job*, fixed before and independent of which resume is being scored, satisfying CUPED's pre-experiment requirement.

Worked result. On the reference run the covariate regression achieved `R^2 = 0.549`, i.e. a **54.9% variance reduction**. The effective-sample-size multiplier is

$$\frac{1}{1-R^2} = \frac{1}{1-0.549} \approx 2.22,$$

so the adjusted estimate is as precise as one from roughly **2.22x** the data. We document this deviation from the spec openly because an interviewer *should* ask "why those covariates?" — and the honest, correct answer (constant covariates reduce nothing; the unit is a job) is stronger than a plausible-but-wrong one.

An honest caveat. Because cluster is also the stratifying variable in Section 11, using it as a CUPED covariate removes exactly the between-cluster variance that the per-cluster analysis then re-examines. The overall adjusted ATE remains unbiased and genuinely tighter, but we would not double-count this variance reduction as if it were independent of the stratified findings — it is the same structure viewed two ways.

Why the reduction equals `R^2`. Minimizing the variance of `d - thetaX` over `theta` is a one-line calculus problem:

$$\mathrm{Var}(d - \theta X) = \mathrm{Var}(d) - 2\theta\,\mathrm{Cov}(d, X) + \theta^2\,\mathrm{Var}(X),$$

minimized at `theta* = Cov(d,X)/Var(X)`; substituting back leaves `Var(d).(1 - rho^2)`, so the fractional variance removed is exactly `rho^2 = R^2`. With several covariates `rho^2` becomes the regression's multiple `R^2`.

What 54.9% buys, concretely. A confidence interval's half-width scales with the standard deviation, i.e. with `sqrt Var`. Cutting variance by 54.9% leaves `sqrt (1 - 0.549) = sqrt 0.451 ~= 0.67` of the original width — a roughly **33% narrower** interval from the very same 2,000 jobs. That is the practical payoff of residualizing on cluster structure: a sharper estimate at zero additional data cost.

9. Sequential Testing (mSPRT)

A fixed-horizon p-value is only valid if you look exactly once. The moment an analyst peeks at accumulating data and stops when it looks significant, the true Type I error inflates far above the nominal alpha. The **mixture Sequential Probability Ratio Test** (Robbins, 1970) solves this by yielding an **always-valid** p-value: you may peek at any sample size, any number of times, without inflating Type I error. With a normal mixture prior `N(0, tau^2)` on the alternative mean and per-job variance estimate `sigma^2`, the mixture likelihood ratio after `n` observations is

$$\Lambda_n = \sqrt{\frac{\sigma^2}{\sigma^2 + n\tau^2}}\,\exp\!\left(\frac{n^2\,\tau^2\,\bar{d}_n^2}{2\,\sigma^2(\sigma^2 + n\tau^2)}\right), \qquad p_n = \min\!\Big(1,\ \min_{m \le n} \tfrac{1}{\Lambda_m}\Big).$$

Why it is always-valid. Under `H_0`, the mixture statistic `Lambda_n` is a non-negative martingale with `E[Lambda_n] = 1`. Ville's inequality then bounds the probability that the process *ever* exceeds `1/alpha` by `alpha`, so `1/Lambda_n` is a valid p-value at every `n` simultaneously — the running minimum in the formula above is what makes peeking safe. This is a different guarantee from **alpha-spending** designs (Pocock, O'Brien-Fleming), which also permit interim looks but only at a *pre-specified* schedule and budget; mSPRT places no constraint on when or how often you look. The mixture width `tau` trades early sensitivity against asymptotic efficiency — a wider prior detects large effects sooner; a narrower one is more powerful against small effects.

In the static-snapshot product all 2,000 jobs are scored at once, so mSPRT is not an operational stopping rule here; we display it as a "valid at any sample size" alternative p-value and plot its trajectory across the accumulating jobs. On the reference run it returns an always-valid `p ~= 3e-67` (reject). Its real value in this product is conceptual honesty about peeking — and it is a genuine differentiator in a Product Analyst interview, where most candidates have never implemented one. We verify the implementation by simulation: across many null runs the empirical "ever reject" rate stays at or below alpha.

How bad is peeking, concretely? A fixed-horizon 0.05 test is only honest at a single predetermined look. Check the data repeatedly and stop the moment it first crosses significance, and the true Type I error climbs fast: naively, `k` looks inflate it toward `1 - 0.95^k` (~= 0.14 at three looks, ~= 0.23 at five). Sequential looks are correlated, so the real figure is somewhat lower, but the direction is unambiguous — and under unbounded continuous monitoring a fixed-horizon test rejects a true null with probability approaching **1**. mSPRT is the principled escape: its always-valid guarantee holds no matter how many times, or when, you look.

Choosing the mixture width. The prior variance `tau^2` is the one tuning knob: a wide `tau` spreads prior mass over large alternatives and so detects a big true effect in very few observations, while a narrow `tau` concentrates near the null — more powerful against small effects but slower to fire. We set `tau` to a modest multiple of the observed delta scale so the test is sensitive across the range of effects real resume pairs produce. Because the always-valid guarantee holds for *any* fixed `tau`, this choice trades one sensitivity profile for another; it never trades away validity.

10. Bayesian Framing

We complement the frequentist machinery with a conjugate Bayesian model that answers a question users find intuitive: *what is the probability B beats A on a randomly chosen job?* Treat each job as a Bernoulli trial `[d_i > 0]`, let `k` be the number of B-wins out of `N`:

$$p \sim \text{Beta}(1, 1), \qquad p \mid \text{data} \sim \text{Beta}(1 + k,\ 1 + N - k).$$

The `Beta(1,1)` prior is uniform on [0, 1] — maximally non-committal about the win rate. (A Jeffreys `Beta(0.5, 0.5)` prior is an equally defensible reference choice; at N = 2,000 the data overwhelm either prior, so the posterior is effectively likelihood-driven and the choice is immaterial.) Conjugacy means the posterior is again a Beta, with a closed-form mean `(1+k)/(2+N)` and exact quantiles for the credible interval — no MCMC required.

Worked result. On the reference run `k = 590` of `N = 2,000` jobs favored B, so the posterior is `Beta(591, 1411)` with mean

$$\frac{1 + 590}{2 + 2000} = \frac{591}{2002} \approx 0.295,$$

a 95% credible interval of roughly [0.275, 0.315], and `P(p > 0.5 | data) ~= 0` — B beats A on under 30% of jobs and the probability it wins *more often than not* is negligible. The Bayesian posterior and the frequentist test agree (both say A is the global winner), as they nearly always will at N = 2,000, but the posterior communicates **uncertainty as probability** — "B wins about 30% of jobs, and we are confident it is below half" — which is far more natural for a non-technical user than a p-value of 7e-75. Note this win-*rate* question (Section 10) is distinct from the win-*magnitude* question (Sections 6–8): a resume could win a slim majority of jobs by tiny margins yet lose on average, or vice versa, so we report both.

The posterior's spread, and what it is for. The Beta posterior carries its own variance,

$$\text{Var}(p \mid \text{data}) = \frac{(1 + k)(1 + N - k)}{(2 + N)^2\,(3 + N)},$$

which at `k = 590, N = 2,000` is tiny — the credible interval [0.275, 0.315] is barely four points wide. The value of the Bayesian layer is not a different verdict (it agrees with the frequentist test) but a different *language*: a product manager can act on "B wins about 30% of jobs, near-certainly under half" without parsing a p-value of 7e-75. And because the win-rate question is orthogonal to the win-magnitude question, reporting the posterior alongside the mean-delta test guards against the trap of a resume that wins many jobs by a sliver yet loses on average.

11. Multiple Comparisons

Running a paired test within each of the 8 clusters produces 8 p-values. If each is judged at alpha = 0.05, the probability of at least one false positive under a global null is `1 - 0.95^8 ~= 0.34` — roughly a one-in-three chance of crying wolf somewhere. We therefore report two corrections, controlling two different error notions:

- **Bonferroni** — controls the *family-wise error rate* (the probability of *any* false positive) by testing each cluster at `alpha/8 = 0.00625`. It is conservative and appropriate when even a single false claim is costly.
- **Benjamini-Hochberg FDR** — controls the *false-discovery rate* (the expected fraction of claimed effects that are false) at 0.05. The procedure sorts the 8 p-values ascending and finds the largest `i` with `p_(i) <= (i/8).0.05`, rejecting all hypotheses up to it. It is less conservative and more appropriate here, because we genuinely *expect* several clusters to show real effects and would rather tolerate a small, controlled fraction of false discoveries than miss true ones.

Each cluster is flagged as a win for B (green), a win for A (red), or not significant (grey) **after** correction, and rendered as a forest plot with 95% CI whiskers. Reporting both corrections lets a strict reader use Bonferroni and a discovery-oriented reader use FDR without re-running anything.

Worked BH procedure. Suppose the 8 clusters yield p-values sorted ascending as `p_(1) … p_(8)`. BH compares each `p_(i)` to its threshold `(i/8).0.05`: 0.00625, 0.0125, 0.01875, 0.025, 0.03125, 0.0375, 0.04375, 0.05. It finds the largest rank `i` whose p-value still falls under its threshold and rejects every hypothesis up to and including that rank. So a cluster with raw `p = 0.03` — which Bonferroni rejects ($0.03 > 0.00625$) — can still be declared a real effect under BH if enough other clusters are even more significant, because the step-up threshold has risen to meet it. That is the deliberate trade: BH recovers true role-level effects that Bonferroni's flat `alpha/8` would discard, at the cost of a controlled expected false-discovery fraction.

The multiplicity problem, quantified. Testing `m` independent hypotheses each at alpha = 0.05 inflates the family-wise error rate as `1 - 0.95^m`:

Tests `m`	2	4	8	16
FWER at alpha = 0.05	0.10	0.19	0.34	0.56

At our `m = 8` clusters the uncorrected chance of at least one false positive is about one-in-three — unacceptable for a verdict a user will act on — which is exactly what the two corrections above exist to control.

12. Decision Framework

The single recommendation combines three independent lines of evidence:

```
significant := (primary_p < 0.05) AND (bootstrap_BCa_CI excludes 0)
winner      := B if significant and mean_delta > 0
               A if significant and mean_delta < 0
               tie otherwise
confidence  := high     if primary_p < 0.01 and max(P(B>A),1-P(B>A)) > 0.99
               moderate if primary_p < 0.05 and ...> 0.95
               low      otherwise
```

Requiring **both** a significant test **and** a bootstrap interval that excludes zero guards against any single method's fragility: the t-test can be fooled by non-normality, the percentile bootstrap by skew, so we demand agreement between a parametric/rank test and a resampling interval before declaring a winner. Confidence is then graded by combining frequentist and Bayesian extremity, and the per-cluster table tells the user *where* to trust the headline.

When the methods disagree. At N = 2,000 the frequentist test and the Bayesian posterior almost always point the same way, but the framework is explicit about the exceptions — typically a borderline effect where

the test clears $p < 0.05$ yet the win-rate posterior straddles 0.5, or the bootstrap interval grazes zero. There the conjunction rule (significant *and* CI excludes zero) and the confidence grading both pull the verdict down to *low*, and the product says "leaning, not conclusive" rather than forcing a winner. Disagreement among methods is treated as information about uncertainty, not a contradiction to be hidden.

Worked example (reference run on a sample resume pair). Comparing a DevOps-flavored Resume A against a Data-Science-flavored Resume B over the 2,000-job corpus produced: Resume A wins overall by **2.08 points** (BCa 95% CI [1.84, 2.30]). The normality gate rejected normality, so the engine auto-selected the **Wilcoxon** test ($p \sim= 7e\text{-}75$); Cohen's d = -0.40, achieved power ~= 1.00. CUPED cut variance by **54.9%** (effective N x2.22), driven by between-cluster structure. mSPRT's always-valid $p \sim= 3e\text{-}67$ (reject). The Bayesian posterior put the per-job B-win rate at ~= 0.295 with `P(p > 0.5) ~= 0`. Crucially, the **per-cluster** view inverted the headline where it counts: **B won Machine Learning / AI (+3.87 pts)** and **QA / Testing (+5.24 pts)**, while A dominated Data Engineering, DevOps / SRE / Cloud, Backend Engineering, and the remaining clusters. The resulting recommendation — *"send B for ML and QA roles, send A for infrastructure roles"* — is the entire point of the product, and it is one that no single global number could ever have produced.

What "tie" and "confidence" mean to the user. A *tie* is not a failure of the test — it is a finding: the two variants are statistically indistinguishable on this corpus, so the user can choose on other grounds (length, readability) without leaving match-quality on the table. The graded *confidence* maps directly onto how the UI presents the verdict: a *high* verdict is stated plainly with a colored card; a *moderate* one is shown with a visible caveat; a *low* one is framed as "leaning B, but not conclusive — consider both." Encoding uncertainty into the presentation, rather than collapsing everything to a single yes/no, is what keeps the product honest with a non-technical audience.

Reproducibility and cross-language validation. Every figure in this document is deterministic: the snapshot and its embeddings are committed as Parquet, the bootstrap and clustering are seeded (`seed = 42`), and the embedding model version is pinned, so the same two resumes always yield the same verdict. As an independent check on the implementation, the entire pipeline is reproduced in R (tidyverse, `pwr`, `boot`) and rendered via Quarto; the Python and R results agree to within `1e-8` on the point estimates and to within Monte-Carlo tolerance on the bootstrap intervals. Two implementations in two languages converging on the same numbers is the strongest evidence available that the statistics are coded correctly and not an artifact of one library's defaults.

13. Limitations and Future Work

- **Embedding-as-proxy (construct validity).** Cosine similarity measures textual relevance, not recruiter behavior or hiring outcomes. A higher score is evidence of better keyword/semantic overlap, not a guaranteed callback. The construct we measure ("does this resume's language match this job's language") is a defensible but imperfect stand-in for the construct we care about ("does this resume get interviews").
- **Single resume pair.** Every statistic here is conditional on one specific A/B pair; the variance reduction, effect size, and per-cluster pattern are properties of *that* comparison, not universal constants. Different pairs yield different numbers — which is why the product recomputes everything per upload and the case-study figures are labeled as a reference run, not a benchmark.
- **Snapshot recency.** The corpus is a fixed 2,000-job snapshot; the labor market drifts. A scheduled refresh (inherited from JobAtlas) mitigates but does not eliminate staleness, and cluster boundaries can shift when the snapshot is rebuilt.
- **No recruiter-side validation.** We never observe whether higher-scoring resumes actually get more interviews — the ground truth we would need to validate the proxy. Without it, every result is internally valid (the math is correct) but externally unvalidated (the proxy is unconfirmed).
- **CUPED covariate scope.** The covariates are job-side and modest; a richer set (job seniority, salary band, named-entity skills) could reduce variance further, and the cluster covariate overlaps the stratification (Section 8 caveat).
- **Score comparability across pairs.** Cosine scores are not calibrated to an absolute scale — a "0.64

mean match" is meaningful only *relative* to the other resume on the same corpus, not as a portable grade. Two different users' headline points are not directly comparable, which is why the product never presents the raw score as a standalone quality number, only as an A-vs-B contrast on one shared corpus.

- **Repeated comparisons by one user.** A user who uploads many resume pairs and keeps only the "winning" variant is, in effect, running an uncorrected multiple-comparison search of their own — a cross-run analogue of the within-run multiplicity of Section 11. The product reports each comparison independently and does not currently warn about this selection effect across uploads.
- **Future work.** Calibrate scores against real callback data once available; per-role fine-tuned embeddings; and counterfactual guidance — *"the single edit that would most raise your ML/AI score"* — turning the diagnostic into a prescriptive tool.

14. References

- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences.* 2nd ed.
- Efron, B., Tibshirani, R. (1993). *An Introduction to the Bootstrap.* (BCa intervals)
- Robbins, H. (1970). "Statistical methods related to the law of the iterated logarithm." *Annals of Mathematical Statistics*, 41(5). (mixture SPRT / always-valid inference)
- Johari, R., Pekelis, L., Walsh, D. (2017). "Always Valid Inference: Bringing Sequential Analysis to A/B Testing." *arXiv:1512.04922.*
- Deng, A., Xu, Y., Kohavi, R., Walker, T. (2013). "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data (CUPED)." *WSDM.*
- Kohavi, R., Tang, D., Xu, Y. (2020). *Trustworthy Online Controlled Experiments.* Cambridge University Press.
- Benjamini, Y., Hochberg, Y. (1995). "Controlling the False Discovery Rate." *JRSS-B*, 57(1).
